

UNIT III

Semantic Analysis * Symbol Tables * Runtime Environments

SEMANTIC ANALYSIS

Attribute Grammars | SDD | SDT Schemes | Evaluation Order

SYMBOL TABLE

Structure & Attributes | Operations | Data Structures | Scope Mgmt

RUNTIME ENV

Activation Records | Parameter Passing | Memory Allocation

SCOPE

All Qs & As from Summer 2025 / Winter 2024 / Summer 2024 / 2023

TABLE OF CONTENTS

PART A: SEMANTIC ANALYSIS

1. Introduction to Semantic Analysis - Role, Need, Types of Checks	3
2. Attribute Grammars - Full Theory with Examples	4
3. Syntax Directed Definitions (SDD) - In Depth	6
4. Syntax Directed Translation Schemes (SDT) - In Depth	8
5. S-Attributed and L-Attributed Definitions	10
6. Evaluation Order - Dependency Graphs, Topological Sort	12

PART B: SYMBOL TABLE

7. Symbol Table - Complete Theory, Need, Contents	14
8. Symbol Table Operations - insert, lookup, delete, update	15
9. Data Structures for Symbol Table - All Types with Examples	16
10. Scope Management - Nested Scopes, Block-Structured Languages	18

PART C: RUNTIME ENVIRONMENT

11. Runtime Memory Organization - Full Layout	20
12. Activation Records - All Fields Explained in Detail	21
13. Parameter Passing - All 4 Mechanisms with Examples	24
14. Memory Allocation - Static, Stack, Heap in Depth	27

PART D: SOLVED PYQs

15. All Solved Questions - Q5 & Q6 across all years with full answers	30
16. Quick Revision Chart	45

PART A - SEMANTIC ANALYSIS

1. INTRODUCTION TO SEMANTIC ANALYSIS

Semantic Analysis is the **third major phase** of a compiler, coming after Lexical Analysis and Syntax Analysis. While Lexical Analysis deals with tokens (is each word valid?) and Syntax Analysis deals with grammar (are words arranged in a grammatical structure?), Semantic Analysis deals with **meaning** -- does the program actually make sense logically and type-theoretically?

Consider this analogy: in English, the sentence "Colorless green ideas sleep furiously" is syntactically correct but semantically nonsensical. Similarly, `int x = "hello";` is syntactically correct C code but semantically wrong because a string literal cannot be assigned to an int. Semantic analysis catches exactly such errors.

1.1 Position of Semantic Analysis in the Compiler Pipeline:

Phase #	Phase Name	Input	Output	Key Tool
1	Lexical Analysis	Source code (characters)	Token stream	Finite Automaton / DFA
2	Syntax Analysis	Token stream	Parse tree / AST	Push-down automaton / Parser
3	Semantic Analysis	Parse tree + Symbol Table	Annotated AST	Attribute Grammar / Type checker
4	Intermediate Code	Annotated AST	Three-address code	SDT Schemes
5	Optimization	TAC	Optimized TAC	Data Flow Analysis
6	Code Generation	Optimized code	Target machine code	Register allocator

1.2 What Does Semantic Analysis Check?

Semantic analysis is responsible for a wide range of correctness checks that go beyond pure syntax:

Type Checking:

Every operator must receive operands of compatible types. The compiler infers or checks types of every expression.

Scope Checking / Name Resolution:

Every identifier must be declared before it is used. The compiler checks that a name is visible in the current scope.

Arity Checking:

Function calls must supply the correct number of arguments matching the function's parameter list.

Flow-of-Control Checking:

break and continue statements must appear inside loops. return statements must match the function's declared return type.

Initialization Checking:

Some compilers warn when a variable is used before being assigned a value.

Overloading Resolution:

In C++ and Java, the compiler uses argument types to determine which overloaded function version to call.

KEY INSIGHT

Semantic analysis is the phase where the compiler builds most of the symbol table and uses it.

Unlike lexical and syntax errors which prevent further compilation, many semantic errors can be reported and compilation can continue to find more errors.

2. ATTRIBUTE GRAMMARS - COMPLETE THEORY

An **Attribute Grammar** is the formal mathematical framework used to specify the semantics of a programming language via a context-free grammar. The idea was introduced by **Donald Knuth in 1968** and is the foundational formalism underlying all practical semantic specification in compilers today.

The core idea: each grammar symbol can have **attributes** -- named values associated with instances of that symbol in the parse tree. Each grammar production has associated **semantic rules** that define how attribute values are computed.

2.1 Formal Definition:

FORMAL DEFINITION

An Attribute Grammar is a triple $AG = (G, A, R)$ where:

$G = (N, T, P, S)$ is a context-free grammar with nonterminals N , terminals T , productions P , start symbol S .

$A = \{A(X) \mid X \text{ is a grammar symbol}\}$ is a finite set of attributes for each symbol.

$R = \{R(p) \mid p \text{ is a production in } P\}$ is a set of semantic rules for each production.

2.2 Types of Attributes - Synthesized vs Inherited:

Synthesized Attributes:

A synthesized attribute of a nonterminal A in a production $A \rightarrow \alpha$ is one whose value is computed from attribute values of children of A in the parse tree, and/or attribute values of terminals in the production body.

Think of it as information **flowing upward** through the parse tree -- from leaves toward the root.

Examples of synthesized attributes:

- $E.val$ in an expression grammar (computed from children $E1.val$ and $T.val$)
- $E.type$ in a type-inference grammar (type of expression derived from operand types)
- $T.place$ in a three-address code generator (temp variable name for this expression)
- $digit.lexval$ (given directly by the lexer -- value of the digit token)

Inherited Attributes:

An inherited attribute of a nonterminal B at position j in a production $A \rightarrow X_1 X_2 \dots X_n$ is one whose value is computed from attribute values of A (the parent), and/or attribute values of siblings to the LEFT of B .

Think of it as information **flowing downward or sideways** -- from parent to children, or from left siblings to right siblings.

Examples of inherited attributes:

- L.in in a declaration grammar (type inherited from $D \rightarrow T L$, where T.type flows to L.in)
- T.inh in a translation grammar (inheriting context from parent or left sibling)
- indent.level in a pretty-printer (current indentation depth passed downward)
- scope.level in a semantic analyzer (current nesting depth passed to all children)

2.3 Attribute Grammar Example -- Expression Evaluation:

Production Rule	Semantic Rule (Synthesized .val)
$L \rightarrow E \text{ newline}$	$L.val = E.val ; \text{print}(L.val)$
$E \rightarrow E1 + T$	$E.val = E1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T1 * F$	$T.val = T1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Worked Example: Parsing and Evaluating $3 + 5 * 2$

1. Lexer gives: digit(3), +, digit(5), *, digit(2)
2. $F \rightarrow \text{digit}(3) : F.val = \text{digit.lexval} = 3$
3. $T \rightarrow F : T.val = F.val = 3$
4. $E \rightarrow T : E.val = T.val = 3$ (left subtree done)
5. $F \rightarrow \text{digit}(5) : F.val = 5$
6. $T1 \rightarrow F : T1.val = 5$
7. $F \rightarrow \text{digit}(2) : F.val = 2$
8. $T \rightarrow T1 * F : T.val = T1.val * F.val = 5 * 2 = 10$
9. $E \rightarrow E1 + T : E.val = E1.val + T.val = 3 + 10 = 13$
10. $L \rightarrow E \text{ newline} : L.val = 13 \rightarrow \text{print}(13)$

The final output is **13**. Values are computed bottom-up -- leaves first, then their parents.

2.4 Attribute Grammar Example -- Type Propagation (Inherited):

Production Rule	Semantic Rule
$D \rightarrow T L$	$L.in = T.type$ (inherit type into list)
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{float}$	$T.type = \text{real}$
$L \rightarrow L1 , id$	$L1.in = L.in ; \text{addtype}(\text{id.entry}, L.in)$
$L \rightarrow id$	$\text{addtype}(\text{id.entry}, L.in)$

For **float x, y, z**: T.type = real -> L.in = real -> addtype(z, real) -> addtype(y, real) -> addtype(x, real). All three variables get type 'real' in the symbol table.

3. SYNTAX DIRECTED DEFINITIONS (SDD) - IN DEPTH

A **Syntax Directed Definition (SDD)** is a generalization of a context-free grammar in which each grammar symbol has a set of attributes and each production has a set of semantic rules for computing attribute values. An SDD is a **pure specification** -- it tells us WHAT to compute, but not HOW or in WHAT ORDER to evaluate those rules.

FORMAL DEFINITION

An SDD consists of: (1) A CFG. (2) For each grammar symbol X , a set of attributes $A(X)$. (3) For each production p : $A \rightarrow \alpha$, a set of semantic rules of the form $b := f(c_1, c_2, \dots, c_k)$ where b is an attribute of some grammar symbol in the production and $c_1..c_k$ are attributes of grammar symbols in the production.

Restriction: if b is an attribute of A (LHS), it must be synthesized. If b is an attribute of a symbol in α (RHS), it must be inherited.

3.1 SDD for Type Checking of Expressions:

Production	Semantic Rule for .type
$E \rightarrow E_1 + E_2$	$E.type = (E_1.type == int \text{ AND } E_2.type == int) ? int : type_error$
$E \rightarrow E_1 * E_2$	$E.type = (E_1.type == int \text{ AND } E_2.type == int) ? int : type_error$
$E \rightarrow E_1 \text{ and } E_2$	$E.type = (E_1.type == bool \text{ AND } E_2.type == bool) ? bool : type_error$
$E \rightarrow E_1 < E_2$	$E.type = (E_1.type == E_2.type) ? bool : type_error$
$E \rightarrow id$	$E.type = lookup(id.name).type$ (from symbol table)
$E \rightarrow num$	$E.type = int$
$E \rightarrow true$	$E.type = bool$
$E \rightarrow false$	$E.type = bool$

If any subexpression has `type_error`, the error propagates upward so that the entire expression is flagged once.

4. SYNTAX DIRECTED TRANSLATION (SDT) SCHEMES - IN DEPTH

A **Syntax Directed Translation Scheme (SDT)** is an SDD with the semantic actions embedded directly inside the production rule body using curly braces { }. SDT schemes are the bridge between specification and implementation.

KEY DISTINCTION

SDD: Semantic rules listed separately from grammar. No execution order specified. Like a mathematical definition.

SDT: Semantic actions { } embedded at SPECIFIC POSITIONS inside production bodies. Position determines WHEN actions run. Like an implementation recipe.

4.1 SDT for Infix to Postfix Translation:

```
E -> E1 + T { print('+') }
E -> T
T -> T1 * F { print('*') }
T -> F
F -> ( E )
F -> id { print(id.name) }
F -> num { print(num.val) }
```

Trace for input: **a + b * c**

```
Reduce F -> id(a) -> prints: 'a'
Reduce T -> F -> (no action)
Reduce F -> id(b) -> prints: 'b'
Reduce T1 -> F -> (no action)
Reduce F -> id(c) -> prints: 'c'
Reduce T -> T1 * F -> prints: '*' output so far: a b c *
Reduce E -> E + T -> prints: '+' final output: a b c * +
```

Output: **a b c * +** which is the correct postfix notation.

4.2 SDT for Boolean Expressions with Backpatching:

Production	SDT Actions
B -> B1 or M B2	backpatch(B1.falselist, M.quad) B.truelist = merge(B1.truelist, B2.truelist) B.falselist = B2.falselist
B -> B1 and M B2	backpatch(B1.truelist, M.quad) B.truelist = B2.truelist B.falselist = merge(B1.falselist, B2.falselist)
B -> not B1	B.truelist = B1.falselist B.falselist = B1.truelist
B -> true	B.truelist = makelist(nextquad) emit('goto _')

B -> false	B.falselist = makelist(nextquad) emit('goto _')
M -> epsilon	M.quad = nextquad

5. S-ATTRIBUTED AND L-ATTRIBUTED DEFINITIONS

Two important subclasses -- S-attributed and L-attributed -- can be evaluated efficiently during a single pass of parsing.

5.1 S-Attributed Definitions:

S-ATTRIBUTED DEFINITION

An SDD is S-attributed if it uses ONLY synthesized attributes.

Every semantic rule computes a synthesized attribute of the left-hand-side nonterminal.

No inherited attributes exist in the grammar.

S-attributed SDDs can be evaluated during a SINGLE BOTTOM-UP (LR) parse.

Evaluation order: POSTORDER traversal of the parse tree.

5.2 L-Attributed Definitions:

L-ATTRIBUTED DEFINITION

An SDD is L-attributed if for every production $A \rightarrow X_1X_2...X_n$, each inherited attribute of X_j depends only on: (1) Inherited attributes of A (the parent), OR (2) Any attributes of $X_1, X_2, ..., X_{j-1}$ (symbols to the LEFT of X_j in the production body).

L-attributed includes ALL S-attributed SDDs as a special case.

L-attributed SDDs can be evaluated in a single LEFT-TO-RIGHT depth-first traversal.

Compatible with LL (top-down) parsers AND LR parsers.

5.3 Comparison Table:

Property	S-Attributed	L-Attributed
Attribute types allowed	Only synthesized	Synthesized + Inherited (with left-dependency)
Evaluation traversal	Postorder (bottom-up)	Left-to-right depth-first
Compatible parsers	LR parsers (LALR, SLR, CLR)	LL parsers; also LR
Can handle inherited?	No	Yes (if dependency is from left only)

Typical use	Expression evaluation, code generation	Type propagation, translation involving context
-------------	--	---

6. EVALUATION ORDER - DEPENDENCY GRAPHS & TOPOLOGICAL SORT

Before we can evaluate attributes, we must determine a valid **order of evaluation**. We represent dependencies formally using a **Dependency Graph**.

6.1 Dependency Graph Construction:

- Create a node in the dependency graph for each attribute instance in the parse tree
- For each semantic rule $b := f(c_1, c_2, \dots, c_k)$, add a directed edge from each c_i to b
- If the resulting dependency graph is a DAG, a valid evaluation order exists
- If the dependency graph has a CYCLE, the SDD is circular and CANNOT be evaluated

6.2 Methods of Evaluating Attributes:

Method	Description	Advantages	Limitations
Parse-tree based	Build complete parse tree, construct dependency graph, topologically sort, evaluate	Works for any non-circular SDD	Requires full parse tree in memory; two passes
Rule-based	Compiler-compiler analyzes grammar and determines fixed evaluation order	Fast at runtime; no dependency graph needed	Order determined statically
Oblivious (postorder)	Always evaluate in postorder regardless of actual dependencies	Simplest; works for all S-attributed SDDs	Only correct for S-attributed
On-the-fly (single-pass)	Evaluate attributes as parsing proceeds	Most efficient; single pass	Only works for L-attributed SDDs

EXAM TIP

For S-attributed SDDs: evaluated bottom-up in postorder during LR parsing using a parallel value stack.

For L-attributed SDDs: evaluated left-to-right in a single pass during LL/recursive-descent parsing, with inherited attributes passed as parameters and synthesized attributes returned as values.

PART B - SYMBOL TABLE

7. SYMBOL TABLE - COMPLETE THEORY

DEFINITION

A Symbol Table is a data structure created and maintained by the compiler to store information about every identifier (variable name, function name, class name, etc.) encountered in the source program.

It is used and updated by virtually EVERY phase of the compiler -- the scanner creates initial entries, the parser adds structural information, the semantic analyzer adds types and checks them, and the code generator uses it to allocate memory and resolve addresses.

7.1 Information Stored in a Symbol Table Entry:

Field Name	Data Type	Purpose / Contents	Example Value
Name	String	The identifier as written in source code	'totalAmount'
Kind	Enum	Variable, Function, Procedure, Parameter, Type, Label	Variable
Type	Type descriptor	Data type -- int, float, char, array, pointer, struct, function sig	int[]
Scope level	Integer	Nesting depth: 0=global, 1=function-level, 2=first nested block	2
Storage class	Enum	auto, static, extern, register (in C); local, global, heap	auto
Memory address	Address or offset	Absolute address (global), stack frame offset (local)	offset 16
Array dimensions	Integer array	For array types: number of dimensions and bounds per dimension	[3][4]
Function info	Record	Return type, number of parameters, parameter types list	int, 2 params
Line number(s)	Integer list	Lines where declared and used -- for error messages	declared: 12

7.2 Symbol Table Usage by Each Compiler Phase:

Compiler Phase		How Symbol Table is Used
Lexical Analyzer		Looks up each identifier; if new, creates an entry. Returns token type.
Syntax Analyzer		Inserts declarations into symbol table as it recognizes them.
Semantic Analyzer		Fills in type information; performs type checking; resolves scopes; detects semantic errors.
Intermediate Generator	Code	Reads address/offset info; generates TAC using symbol table entries.
Code Generator		Allocates registers and memory; uses offsets for load/store instructions.

8. SYMBOL TABLE OPERATIONS

insert(name, attribute_list)

Adds a new identifier to the symbol table with its associated attributes.

- Called when parser/semantic analyzer encounters a declaration
- Must check for duplicate declarations in the same scope (error if found)
- Example: parsing 'int counter = 0;' inserts (counter, type=int, val=0, scope=current)

lookup(name)

Searches for an identifier and returns its symbol table entry (or null/error if not found).

- Called whenever an identifier is used in an expression or statement
- Must search from CURRENT (innermost) scope outward to global scope
- The most frequently called operation -- must be as fast as possible (hence hash tables)

delete(name) / delete_scope(scope_level)

Removes an identifier or all identifiers in a scope from the symbol table.

- Called when a block/scope is exited -- all local variables go out of scope
- In a stack-of-tables design, implemented by simply popping the top table

update(name, attribute, value)

Modifies an existing entry -- changes one or more attribute values.

- Used when new information is learned about an already-declared identifier
- Example: fill in the memory address/offset once memory layout is determined

9. DATA STRUCTURES FOR SYMBOL TABLE - ALL TYPES

9.1 Linear List:

The simplest implementation. Identifiers stored sequentially. Insertion appends to end; lookup scans from beginning.

- **Insert:** $O(1)$ -- append at end (but must check for duplicates: $O(n)$)
- **Lookup:** $O(n)$ -- linear scan -- very slow for large programs
- **Suitable for:** Very small programs, macro processors with fewer than 20 entries

9.2 Hash Table with Chaining (Most Common in Production Compilers):

A hash function maps an identifier name to an integer index (bucket). Each bucket is the head of a linked list containing all entries that hash to that bucket. $O(1)$ average-case performance for insert and lookup.

```
// Simple hash function:
int hash(char *name, int size) {
    int h = 0;
    while (*name) h += *name++;
    return h % size;
}

// Polynomial rolling hash (better distribution):
int hash(char *name, int size) {
    int h = 0;
    while (*name) h = (h * 31 + *name++) % size;
    return h;
}
```

- **Insert:** $O(1)$ amortized average -- compute hash, prepend to chain
- **Lookup:** $O(1)$ amortized average -- $O(n)$ worst case if all names collide
- **Table size:** Should be a prime number for better distribution (e.g., 211, 509, 1021)
- **Used in:** GCC, LLVM/Clang, Java VM, virtually all production compilers

9.3 Binary Search Tree (BST):

Identifiers stored in a BST keyed on name. $O(\log n)$ average for balanced trees.

- **Insert/Lookup/Delete:** $O(\log n)$ for balanced BST (AVL or Red-Black tree)
- **Advantage:** Inorder traversal gives identifiers in alphabetical order -- useful for cross-reference listings

9.4 Stack of Symbol Tables (Block-Structured Languages):

For block-structured languages that support nested scopes, we maintain a **stack of symbol tables** -- one table per active scope level. Entering a block pushes a new table; exiting pops it.

```
int a = 1; // Table_0 (global): {a: int}
void f(int x) { // PUSH Table_1: {x:int}
```

```

int a = 10; // Table_1: {x:int, a:int} -- hides global a
{ // PUSH Table_2: {}
int b = 20; // Table_2: {b:int}
a = a + b; // lookup(a): Table_2 no, Table_1 YES -> local a
} // POP Table_2
} // POP Table_1

```

Data Structure	Insert	Lookup	Delete	Best Use Case
Linear List	$O(1)$	$O(n)$	$O(n)$	Tiny programs < 20 identifiers
Hash Table	$O(1)^*$	$O(1)^*$	$O(1)^*$	Production compilers (most common)
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	When sorted output needed
Scope Stack+Hash	$O(1)^*$	$O(k)^*$	$O(1)^*$	Block-structured languages

** amortized/average case k = scope nesting depth*

10. SCOPE MANAGEMENT IN DETAIL

Scope is the region of a program within which an identifier declaration is valid (visible).

10.1 Scope Rules:

- **Static (Lexical) Scope:** The scope of an identifier is determined by the textual structure of the program. Used by C, Java, Python, Pascal, most modern languages.
- **Dynamic Scope:** The scope is determined at runtime based on the sequence of procedure calls. Used historically by early Lisp dialects.
- **Name hiding (shadowing):** An inner declaration of the same name hides the outer declaration within the inner scope.

10.2 Scope in C -- Detailed Rules:

- Global scope: declared outside all functions -- visible throughout the entire translation unit
- Block scope: variables declared inside {} -- visible from declaration to the closing }
- File scope with static: visible only in the current source file (no external linkage)

PART C - RUNTIME ENVIRONMENT

11. RUNTIME MEMORY ORGANIZATION

The **runtime environment** is the organization of memory and the set of support routines that manage program execution.

11.1 Memory Regions:

Region	Location	Contents	Allocation Time
Code Segment (Text)	Low addresses (fixed)	Compiled machine instructions; constants	Compile/load time
Static Segment Data	Above code	Global variables, static local variables	Compile/load time
Heap	Above static; grows upward	Dynamically allocated objects (malloc/new)	Runtime (explicit)
Stack	High addresses; grows downward	Activation records for each active procedure call	Runtime (automatic)

11.2 Memory Layout Diagram:

```
High addresses
+=====+
| STACK (grows down) |
| [activation records] |
| |
| v v v |
| |
| (unused gap) |
| |
| ^ ^ ^ |
| HEAP (grows up) |
| [malloc/new objects] |
+=====+
| BSS Segment |
| [uninitialized globals] |
+=====+
| Data Segment |
| [initialized globals] |
+=====+
| Text / Code Segment |
| [machine instructions] |
+=====+
Low addresses
```

12. ACTIVATION RECORDS - ALL FIELDS EXPLAINED

When a procedure P is **called**, the runtime system allocates a block of memory on the stack called the **Activation Record** (also called a **Stack Frame**). When P **returns**, its activation record is deallocated.

Return Value:

Space where P deposits its return value before returning. On some architectures, the return value is in a register (EAX/RAX) rather than the stack.

- For small values (int, pointer): often in a register for efficiency
- For large values (structs): caller allocates space and passes pointer to it

Actual Parameters:

The values (or addresses) of arguments passed by the caller to P.

- Call by value: copies of values are stored here
- Call by reference: addresses of caller's variables are stored here

Control Link (Dynamic Link):

A pointer to the activation record of the CALLER. Used to restore the frame pointer (FP) when P returns.

- Always present in every activation record
- On P's return: FP = control_link (restores caller's frame pointer)

Access Link (Static Link):

A pointer to the activation record of P's lexical parent. Used to access non-local variables in nested procedures.

- Only present in languages that allow nested procedure definitions (Pascal, Ada)
- Not present in C (which does not allow nested functions)

Saved Machine State:

Saved values of machine registers that P will overwrite but must restore before returning. Also includes the saved PC (return address).

- Return address: placed by the CALL instruction; restored by RETURN instruction
- Callee-saved registers: P saves any register it uses that must be preserved

Local Variables:

Variables declared locally within P. Each has a fixed offset from the frame pointer, computed at compile time.

- Only exist for the duration of P's activation
- Multiple recursive activations each have their own separate local variables

Temporaries:

Temporary values generated during expression evaluation that do not fit in registers.

- Compiler manages allocation of temporaries at compile time

12.1 Activation Record Layout Diagram:

[Caller's frame]
+-----+
Actual parameters (passed to P) <- pushed by CALLER
+-----+
Return value (space for result) <- read by CALLER after return
+-----+ <- SP before call
[P's activation record begins here]
+-----+
Return address (saved PC) <- pushed by CALL instruction
+-----+
Control link (caller's FP) <- pushed on entry to P
+-----+ <- FP (frame pointer) = this address
Access link (static parent's FP) (only in nested procedure languages)
+-----+
Saved registers
+-----+
Local variables e.g., [FP-4]=a, [FP-8]=b
+-----+
Temporaries compiler-generated t1, t2, ...
+-----+ <- SP during P's execution

13. PARAMETER PASSING MECHANISMS - ALL 4 TYPES

13.1 Call by Value:

CALL BY VALUE

The caller evaluates the actual parameter expression and passes only the RESULTING VALUE.

A fresh COPY of this value is created in the callee's activation record.

Any modifications to the formal parameter inside the procedure affect ONLY the local copy.

The caller's original variable is COMPLETELY UNAFFECTED.

```
void increment(int x) { // x is a copy of the actual parameter
x = x + 1; // modifies the LOCAL copy only
}

int main() {
int a = 10;
increment(a); // passes value 10; a copy is made
printf('%d', a); // prints 10 -- a is UNCHANGED!
}
```

- **Used in:** C (default for all types), Java (primitives)
- **Advantages:** Safe -- caller's data protected; simple to reason about
- **Disadvantages:** Expensive for large structures; cannot return multiple values through parameters

13.2 Call by Reference:

CALL BY REFERENCE

The caller passes the ADDRESS (L-value) of the actual parameter variable.

The formal parameter becomes an ALIAS (another name) for the caller's actual variable.

Any modification to the formal parameter DIRECTLY MODIFIES the caller's variable.

There is no copying -- both formal and actual refer to the same memory location.

```
void swap(int &x;, int &y;) { // x,y are references (aliases)
int temp = x;
x = y; // directly modifies caller's first variable
y = temp; // directly modifies caller's second variable
}
```

```

}

int main() {
int a = 5, b = 10;
swap(a, b);
printf('%d %d', a, b); // prints: 10 5 -- BOTH CHANGED!
}

```

- **Used in:** C++ (& parameters), Fortran (default), C# (ref keyword), Pascal (VAR parameters)
- **Advantages:** Efficient -- no copying; enables procedures to return multiple values
- **Disadvantages:** Aliasing problem -- unexpected side effects; harder to optimize

13.3 Call by Value-Result (Copy-in Copy-out):

CALL BY VALUE-RESULT

Phase 1 -- Copy-in: At the START of the call, the actual parameter's value is COPIED into the formal parameter.

Phase 2 -- Procedure executes on its LOCAL COPY.

Phase 3 -- Copy-out: At the END of the call, the final value of the formal parameter is COPIED BACK to the actual parameter location.

The caller's variable is affected -- but only once, at the end, not during execution.

- **Used in:** Ada (IN OUT mode), some Fortran implementations
- **Differs from call by reference when:** aliasing occurs -- changes are batched and written back at end
- **Advantages:** Suitable for remote procedure calls where continuous pointer access is impossible

13.4 Call by Name:

CALL BY NAME

The actual parameter expression (not its value, not its address) is TEXTUALLY SUBSTITUTED for the formal parameter.

The expression is RE-EVALUATED every time the formal parameter is accessed inside the procedure.

Introduced in ALGOL 60. Implemented using THUNKS (closures capturing the expression and its environment).

- **Used in:** ALGOL 60 (historically); Haskell's lazy evaluation is similar
- **Implementation:** Thunk -- a parameterless closure representing the unevaluated argument
- **Disadvantages:** Very expensive (re-evaluates expression on each access); non-intuitive aliasing behavior

Summary Comparison Table:

Mechanism	What is passed	Caller's var affected?	Used in
Call by Value	Copy of value	Never	C, Java primitives
Call by Reference	Address of var	Always (immediately)	C++, Fortran, C# ref
Call by Value-Result	Value (copy both ways)	On return only	Ada IN OUT
Call by Name	Unevaluated expression	Possible (aliasing)	Algol 60, Haskell

14. STORAGE / MEMORY ALLOCATION STRATEGIES

14.1 Static Allocation:

STATIC ALLOCATION

Memory is allocated at COMPILE TIME. The compiler assigns a FIXED, permanent address to each variable.

No allocation or deallocation occurs at runtime -- variables exist for the ENTIRE duration of program execution.

- Global variables (declared outside any function)
- Static local variables (declared with 'static' keyword in C)
- String literals, global constants
- In Fortran 77: ALL local variables (no recursion support)
- **Advantages:** Zero runtime overhead; addresses known at compile time; simplest to implement
- **Limitations:** Cannot support recursion; all data sizes must be known at compile time

14.2 Stack-Based (Dynamic) Allocation:

STACK ALLOCATION

Activation records are allocated on the RUNTIME STACK using LIFO discipline.

Memory is automatically allocated when a procedure is called and freed when it returns.

Fully supports recursion -- each recursive call gets its own separate activation record with its own locals.

- **Advantages:** Supports recursion; efficient $O(1)$ allocation/deallocation; no fragmentation
- **Limitations:** Size of each activation record must be known at compile time; lifetime tied to procedure activation

14.3 Heap Allocation:

HEAP ALLOCATION

The heap is a large pool of free memory that programs can request and release in ARBITRARY ORDER at runtime.

Allocation: program calls malloc(size) / new -- heap manager finds a free block of sufficient size.

In managed languages (Java, Python, C#): a Garbage Collector automatically reclaims unreachable objects.

Strategy	How It Works	Pros	Cons
First Fit	Scan free list from beginning; return first block large enough	Fast allocation on average	Fragmentation at start
Best Fit	Return smallest block that fits	Minimizes wasted space	Slow (full scan)
Worst Fit	Return largest available block	Leaves large remaining fragments	Quickly exhausts large blocks
Buddy System	Block sizes are powers of 2; split/merge pairs	Fast coalescing of free blocks	Internal fragmentation

Property	Static	Stack	Heap
Allocation time	Compile/Load time	Runtime (call)	Runtime (explicit)
Deallocation	Never (program end)	Automatic (return)	Manual (free/GC)
Supports recursion?	No	Yes	Yes
Fragmentation	None	None	Yes
Allocation overhead	Zero	Very low O(1)	High (manager + GC)
Use case	Globals, statics	Local vars, params	Dynamic data structures

PART D - ALL SOLVED PAST YEAR QUESTIONS

EXAM COVERAGE

The following solved answers cover ALL Question 5 and Question 6 from:

Summer 2025, Winter 2024, Summer 2024, Winter 2023, Summer 2023.

Each answer is written in exam-ready format with sufficient depth to score full marks.

SUMMER 2025 - Questions 5 and 6

Q.5(a) [7 marks] - Semantic actions / SDT for Boolean expressions

Answer:

Semantic Actions are code fragments embedded within grammar productions that define the meaning (semantics) of each construct. In an SDT scheme, they appear in curly braces { } at specific positions within production bodies.

SDT for Evaluation of Boolean Expressions:

Production	Semantic Rule / Action

B -> B1 or B2	B.val = B1.val OR B2.val
(short-circuit: if B1.val=TRUE, skip B2)	
B -> B1 and B2	B.val = B1.val AND B2.val
B -> not B1	B.val = NOT B1.val
B -> (B1)	B.val = B1.val
B -> E1 relop E2	B.val = apply(relop.op, E1.val, E2.val)
B -> true	B.val = TRUE
B -> false	B.val = FALSE

Worked Example: (x < 5) and (y > 0) with x=3, y=2

Step 1: B1.val = apply('<', 3, 5) = TRUE

Step 2: B2.val = apply('>', 2, 0) = TRUE

Step 3: B.val = TRUE AND TRUE = TRUE

Q.5(b) [7 marks] - Different symbol table organizations

Answer:

The five major organizations are:

- **1. Linear List:** O(n) lookup. Used only for tiny programs.
- **2. Hash Table (chaining):** O(1) average. Industry standard (GCC, LLVM, JVM).
- **3. BST (balanced):** O(log n). Produces sorted output for cross-reference tables.

- **4. Self-Organizing List:** Adapts to hot identifiers via move-to-front strategy.
- **5. Stack of Hash Tables:** For block-structured languages. Correctly implements lexical scoping.

Q.6 [14 marks] - Compute IN and OUT equations for flow graph (Live Variable Analysis)

Answer - Live Variable Analysis (Data Flow Analysis):

A variable v is **live** at a program point p if there exists a path from p to some USE of v along which v is not RE-DEFINED. Used for register allocation and dead code elimination.

Data Flow Equations:

USE[B] = variables used in B BEFORE any definition in B

DEF[B] = variables definitely defined in B BEFORE any use in B

OUT[B] = UNION of IN[S] for all successors S of B

IN[B] = USE[B] UNION (OUT[B] - DEF[B])

Direction: BACKWARD; Meet operator: UNION

Given Flow Graph:

Block B1: (1) $I = 2$ (2) $J = J + 1$ Successor: B2

Block B2: (3) $I = 1$ Successor: B3

Block B3: (4) $J = J + 1$ Successors: B4, B5

Block B4: (5) $J = J - 4$ Successor: B3

Block B5: (exit node) No successors

Block	USE[B]	DEF[B]
B1	{J}	{I, J}
B2	{}	{I}
B3	{J}	{J}
B4	{J}	{J}
B5	{I, J}	{}

Iterative Computation (Backward):

Iteration 1:

B5: OUT={}, IN={I, J}

B4: OUT=IN[B3]={} (initial), IN={J}u({}_J)=J

B3: OUT=IN[B4]uIN[B5]={J}u{I, J}={I, J}, IN={J}u({I, J}_J)=I, J

B2: OUT=IN[B3]={I, J}, IN={}_u({I, J}_I)=J

B1: OUT=IN[B2]={J}, IN={J}u({J}_I, J)=J

Iteration 2 (B4 used stale IN[B3] in iter 1):

B4: OUT=IN[B3]={I, J} (updated), IN={J}u({I, J}_J)=I, J <- CHANGED

Iteration 3 (verify convergence -- no changes):

All values stable -> CONVERGED

Final Result:

Block	IN[B] (Live variables at ENTRY)	OUT[B] (Live variables at EXIT)
B1	{J}	{J}
B2	{J}	{I, J}
B3	{I, J}	{I, J}
B4	{I, J}	{I, J}
B5	{I, J}	{}

Q.5 [14 marks] - 3-Address Code for: while (A>B) or (C>B) do if G<H then x=y+z else x=y-z

Answer:

Label	Instruction	Explanation
L1:	t1 = A > B	Evaluate first part of OR condition
	t2 = C > B	Evaluate second part of OR condition
	t3 = t1 OR t2	Combine: while condition
	if t3 == FALSE goto L5	Exit loop if condition false
L2:	t4 = G < H	Evaluate if-condition
	if t4 == FALSE goto L4	Condition false -> go to else branch
L3:	t5 = y + z	Then branch: compute y+z
	x = t5	Assign to x
	goto L1	Loop back to while test
L4:	t6 = y - z	Else branch: compute y-z
	x = t6	Assign to x
	goto L1	Loop back to while test
L5:	(next statement)	Target of while-exit jump

Q.6(a) [7 marks] - Data structures for symbol table with pros/cons

Data Structure	Pros	Cons
Linear List	Extremely simple to implement; Zero implementation overhead	O(n) search time; Does not scale to large programs
Hash Table (chaining)	O(1) average insert, lookup, delete; Used in all production compilers	O(n) worst-case with collisions; No natural ordering
BST (balanced)	O(log n) guaranteed; Inorder traversal gives sorted listing	Complex implementation; Slower than hash table in practice
Self-Organizing List	Adapts to access patterns; Frequently used identifiers approach O(1)	O(n) worst case; Reorganization overhead on every access
Stack of Hash Tables	Correctly implements block-structured scoping; O(1) scope entry/exit	Lookup may traverse multiple tables; More memory per scope

Q.6(b) [7 marks] - What is an activation record? When is it set up?

Answer:

An **Activation Record** is a block of memory allocated on the runtime stack to hold all information required for a single invocation of a procedure or function. Every time a procedure is called, a fresh activation record is created. Every time the procedure returns, its activation record is destroyed.

Setup Sequence:

- **Caller's actions:** Evaluates actual parameters; pushes them on stack; executes CALL instruction (saves return address, jumps to callee).
- **Callee's prologue:** Saves old FP as control link; sets FP = SP; decrements SP by frame size; saves callee-saved registers.
- **On return (epilogue):** Stores return value; restores saved registers; restores SP = FP; restores FP = control_link; executes RETURN.

Q.5(a) [10 marks] - TAC for: for(i=1;i<50;i=i+1) if(i<10) then a=b+1 else a=c+1**Answer -- Three-Address Code:**

Label	Instruction	Explanation
L0:	i = 1	INITIALIZATION
L1:	t1 = i < 50	CONDITION CHECK
	if t1 == FALSE goto L6	EXIT TEST
	t2 = i < 10	INNER IF condition
	if t2 == FALSE goto L4	BRANCH to else
L2:	t3 = b + 1	THEN BRANCH
	a = t3	ASSIGN a = b+1
	goto L5	SKIP ELSE
L4:	t4 = c + 1	ELSE BRANCH
	a = t4	ASSIGN a = c+1
L5:	t5 = i + 1	UPDATE: i+1
	i = t5	UPDATE: assign new i
	goto L1	LOOP BACK
L6:	(end of for loop)	EXIT TARGET

Q.5(b) [4 marks] - Explain inherited and synthesized attributes

Synthesized Attribute: An attribute whose value is computed from the attributes of its CHILDREN in the parse tree. Information flows UPWARD from leaves to root. Example: $E.val = E1.val + T.val$ (value of E synthesized from children E1 and T).

Inherited Attribute: An attribute whose value is computed from the attributes of its PARENT or LEFT SIBLINGS in the parse tree. Information flows DOWNWARD. Example: $L.in = T.type$ in $D \rightarrow T L$ -- the type is inherited by the identifier list from the type specifier.

Q.6 [14 marks] - Compute IN and OUT for the flow graph (Summer 2024)

B1: a=2, b=a+1 -> USE[B1]={}, DEF[B1]={a, b}

B2: a=1 -> USE[B2]={}, DEF[B2]={a}

B3: b=b+1 -> USE[B3]={b}, DEF[B3]={b}

B4: b=b+7 -> USE[B4]={b}, DEF[B4]={b}

B5: a=a+2 -> USE[B5]={a}, DEF[B5]={a}

Iteration 1 (backward: B5->B4->B3->B2->B1):

B5: OUT={}, IN={a}

B4: OUT=IN[B5]={a}, IN={b}u({a}_b)={a, b}

B3: OUT=IN[B4]={a, b}, IN={b}u({a, b}_b)={a, b}

B2: OUT=IN[B3]={a, b}, IN={}u({a, b}_a)={b}

B1: OUT=IN[B2]={b}, IN={}u({b}_a, b)={}

Iteration 2: All values same -> CONVERGED

Final: IN[B1]={}, OUT[B1]={b} | IN[B2]={b}, OUT[B2]={a, b}

IN[B3]={a, b}, OUT[B3]={a, b} | IN[B4]={a, b}, OUT[B4]={a}

IN[B5]={a}, OUT[B5]={}

Q.5(a) [6 marks] - Sources of error in a compiler

Lexical Errors:

Invalid characters, unterminated strings, malformed numeric literals. Recovery: skip to next delimiter.

Syntax Errors:

Missing semicolons, mismatched parentheses, invalid token in expression. Recovery: error productions, panic mode.

Semantic Errors:

Undeclared variable, type mismatch, wrong number of arguments, return type mismatch.

Logical Errors:

Program compiles and runs but produces incorrect results. CANNOT be caught by compiler.

Runtime Errors:

Division by zero, null pointer dereference, stack overflow, array index out of bounds.

Q.5(b) [8 marks] - Data structure for symbol table in block structured language

The Stack of Symbol Tables Design:

One hash table per active scope, maintained in a stack. Entering a block pushes a new table; exiting pops it. Lookup starts at top (innermost scope) and works downward.

```
int a = 1; // Global scope (table 0): {a:int}
void f(int x) { // ENTER SCOPE -> push table 1: {x:int}
float a = 3.14; // table 1: {x:int, a:float} -- shadows global a
{ // ENTER SCOPE -> push table 2
int b = 99; // table 2: {b:int} -- shadows global b
int c = a; // lookup(a): table2->no, table1->YES (float a)
} // EXIT SCOPE -> pop table 2
} // EXIT SCOPE -> pop table 1
```

Q.6(a) [7 marks] - Memory allocation in procedure call and return

Memory allocation in procedure call/return uses a RUNTIME STACK with LIFO discipline. On call: push activation record. On return: pop activation record. Each recursive call gets its own frame.

Q.7(a) [7 marks] - Different loop optimization techniques

Code Motion:

Move computations that produce the same value every iteration outside the loop. Example: while(i < a*b) -> t=a*b; while(i < t).

Induction Variable Elimination:

Replace expensive multiplications with cheaper additions for loop variables in arithmetic progressions.

Strength Reduction:

Replace expensive operations with cheaper equivalents. Multiply replaced by addition.

Loop Unrolling:

Replicate the loop body multiple times, reducing branch/counter update overhead.

Loop Fusion:

Combine two adjacent loops with the same iteration range into a single loop.

Q.5(a) [7 marks] - Symbol table organization

(Complete discussion in Sections 7-10 above. Summary:)

- **1. Linear List:** Simplest. $O(n)$ lookup. Used only for tiny programs.
- **2. Hash Table (chaining):** Industry standard. $O(1)$ average. Used in GCC, LLVM, JVM.
- **3. BST (balanced):** $O(\log n)$. Produces sorted output. Good for cross-reference tables.
- **4. Self-Organizing List:** Adapts to hot identifiers. Still $O(n)$ worst case.
- **5. Stack of Hash Tables:** For block-structured languages. Correctly implements lexical scoping.

Q.5(b) [7 marks] - SDTS and TAC for: while(a<10 and C>D) if(a<b) then a=a+b else b=a+b

TAC with short-circuit AND evaluation:

Label	Instruction	Explanation
L1:	t1 = a < 10	First part of AND condition
	if t1 == FALSE goto L5	Short-circuit: if first part false, exit loop
	t2 = C > D	Second part of AND condition
	if t2 == FALSE goto L5	If second part false, exit loop
L2:	t3 = a < b	Inner if-condition
	if t3 == FALSE goto L4	Branch to else
L3:	t4 = a + b	Then branch: a+b
	a = t4	a = a+b
	goto L1	Loop back
L4:	t5 = a + b	Else branch: a+b
	b = t5	b = a+b
	goto L1	Loop back
L5:	(exit while loop)	Loop exit target

Q.6(a) [7 marks] - Need of symbol table; data structure for management

Need: Without a symbol table, the compiler cannot perform type checking, detect undeclared variables, allocate memory correctly, distinguish local vs global variables, resolve function call arguments, or generate correct code with proper offsets.

Hash Table Implementation Example:

```
// Hash for 'total':
h = 't'+'o'+'t'+'a'+'l' = 116+111+116+97+108 = 548
bucket = 548 mod 211 = 126
Entry for 'total' placed in bucket[126].
```

Q.6(b) [7 marks] - Run time storage management for call and return

On Call:

- Evaluate and push actual parameter values
- CALL instruction: push return address, jump to callee
- Callee: push current FP (control link), set FP = SP
- Callee: $SP = SP - \text{sizeof}(\text{locals} + \text{temps})$ to allocate locals
- Callee saves callee-saved registers; begins execution

On Return:

- Place return value at designated location (or register EAX/RAX)
- Restore callee-saved registers
- Restore $SP = FP$; Restore $FP = \text{control_link}$ (old frame pointer)
- RETURN instruction: pop return address, jump to it
- Caller reads return value; pops actual parameters if caller's responsibility

QUICK REVISION CHART - EVERYTHING AT A GLANCE

Topic	One-Line Definition	Key Formula / Rule	Exam Freq
Attribute Grammar	CFG extended with attributes and semantic rules	$AG = (G, A, R)$	HIGH
Synthesized Attribute	Computed from children nodes in parse tree	$A.s = f(\text{children})$ -- flows UP	VERY HIGH
Inherited Attribute	Computed from parent or left siblings	$B.i = f(\text{parent/left})$ -- flows DOWN	VERY HIGH
SDD	Specification: defines WHAT to compute via semantic rules	Pure specification, no execution order	HIGH
SDT	SDD with actions $\{ \}$ embedded inside productions	Position of $\{ \}$ determines WHEN action executes	HIGH
S-Attributed	Uses ONLY synthesized attributes	Evaluated in postorder -- LR parsers	HIGH
L-Attributed	Inherited attrs depend only on parent + left siblings	Evaluated left-to-right -- LL parsers	HIGH
Symbol Table	Compiler data structure storing all identifier info	Ops: insert, lookup, delete, update	VERY HIGH
Hash Table (ST)	Maps name $\rightarrow$ bucket via hash function; $O(1)$ avg	$h(\text{name}) = (\text{sum ASCII}) \bmod \text{TABLE_SIZE}$	VERY HIGH
Scope Stack	Stack of tables; push on $\{$; pop on $\}$	lookup starts at top (innermost) scope	HIGH
Activation Record	Stack frame per procedure call on runtime stack	Fields: ret val, params, ctrl link, FP, ret addr, locals, temps	VERY HIGH
Control Link	Pointer to caller's activation record	Used to restore FP on return (dynamic chain)	HIGH
Access Link	Pointer to lexical parent's activation record	Used to access non-local variables (static chain)	MEDIUM
Call by Value	Copy of value passed; caller unaffected	$\text{formal} = \text{copy}(\text{actual})$; changes don't propagate	VERY HIGH
Call by Reference	Address of variable passed; caller affected immediately	$\text{formal} = \&\text{actual}$; $\ast\text{formal}$ changes actual	VERY HIGH

Call by Value-Result	Copy in + copy back on return	formal=actual on call; actual=formal on return	HIGH
Call by Name	Expression substituted textually; re-evaluated each access	Implemented with thunks (closures)	MEDIUM
Static Allocation	Compile-time memory -- globals, static vars	No recursion support; zero runtime overhead	HIGH
Stack Allocation	LIFO runtime stack for activation records	Supports recursion; O(1) alloc/free (SP+/-size)	VERY HIGH
Heap Allocation	Runtime explicit-request dynamic memory	malloc/new; GC; fragmentation possible	HIGH
Live Variable (IN/OUT)	Variable live if used on some path before re-def	$IN[B] = USE[B] \cup (OUT[B] - DEF[B])$; $OUT[B] = uIN[succs]$	VERY HIGH
Three-Address Code	result = op1 operator op2; at most 3 operands	Temps t1,t2,...; Labels L1,L2,...; if cond goto L	VERY HIGH
Backpatching	Fill jump addresses later when target label known	emit('goto _'); later backpatch(list, target)	MEDIUM

IMPORTANT FORMULAS AND RULES

LIVE VARIABLE ANALYSIS (Backward):

$OUT[B] = \text{UNION} \{ IN[S] : S \text{ is a successor of } B \}$

$IN[B] = USE[B] \cup (OUT[B] - DEF[B])$

Initialize: $OUT[exit] = \{\}$, $IN[B] = \{\}$ for all B

Iterate until fixed point (no changes)

REACHING DEFINITIONS (Forward):

$IN[B] = \text{UNION} \{ OUT[P] : P \text{ is a predecessor of } B \}$

$OUT[B] = \text{Gen}[B] \cup (IN[B] - \text{Kill}[B])$

HASH FUNCTION:

$h(\text{name}) = (\text{sum of ASCII values of characters in name}) \bmod \text{TABLE_SIZE}$

TABLE_SIZE should be a prime number (e.g., 211, 509, 1021, 4093)

ACTIVATION RECORD:

$\text{frame_size} = \text{sizeof}(\text{local_vars}) + \text{sizeof}(\text{temporaries}) + \text{sizeof}(\text{saved_registers})$

$\text{SP_new} = \text{SP_old} - \text{frame_size}$ (stack grows downward)

local_var_i is at address $[\text{FP} - \text{offset_i}]$ (compiler computes offset_i at compile time)

END OF UNIT 3 NOTES

This document covers the complete Unit 3 syllabus for Compiler Design (6th Semester B.Tech. CSE, RTM Nagpur University). All topics explained with examples and all PYQs from 2023-2025 are solved.

Topics Covered:

- Attribute Grammars (Synthesized & Inherited)
- SDD -- Syntax Directed Definitions
- SDT -- Syntax Directed Translation Schemes
- S-Attributed and L-Attributed Definitions
- Evaluation Order, Dependency Graphs
- Symbol Table -- Theory, Operations, Data Structures
- Scope Management in Block-Structured Languages
- Runtime Memory Organization
- Activation Records -- All Fields
- Parameter Passing -- Call by Value/Reference/Value-Result/Name
- Memory Allocation -- Static/Stack/Heap
- Solved PYQs: Summer 2025, Winter 2024, Summer 2024, Winter 2023, Summer 2023